Common Core: Phase 3 Physical Design

Lab P1: Logic Synthesis and Gate-Level Simulation

Yosys, Standard Cells and Simulation–Synthesis Mismatches

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Synthesize the multiplier and the FIFO you designed and verified to SkyWater 130 nm standard cells with Yosys, read the synthesis log and statistics, run the V1 testbench on the gate-level netlist, and provoke and explain simulation–synthesis mismatches.

Duration	1.5 weeks (approximately 25–30 hours)
Prerequisites	Labs R1, R2 and V1
Designs	`seq_mul`, `comb_mul` (R1), `sync_fifo` (R2); RTL unchanged
Reused in	P2 (STA of the netlists), P3 (sweeps), T-RTL3 (pipelined multiplier)

Contents

1 Overview **3**

2 What Synthesis Does **4**

2.1 Optimizations You Will See 4

2.2 Why sv2v 5

3 Standard Cells and the Liberty File **5**

3.1 The Library and Its Name 5

3.2 What a Liberty File Contains 5

4 The Synthesis Flow in the Repository **7**

4.1 Generic Yosys Script 7

4.2 `syn.mk`: the `synth` and `gls` Targets 9

5 Reading the Yosys Log and Statistics **10**

5.1 The `stat` Report 11

6 Mapping the Netlist Back to RTL **12**

7 Gate-Level Simulation **13**
7.1 What GLS Finds, and What It Does Not 13
7.2 Which Netlist 14
7.3 Reusing the V1 Testbench 14

8 Mismatch Experiment **15**

9 Requirements / Tasks **16**

10 Repository and Flow **17**

11 Deliverables **17**

12 Assessment **18**

13 Review Questions **18**

14 References **19**

1 Overview

Up to now your designs existed only as RTL that a simulator interprets. A chip is built from a *netlist*: instances of standard cells from a foundry library, connected by wires. Logic synthesis turns the RTL into that netlist. Everything the back-end team does afterwards — placement, clock-tree synthesis, routing, sign-off timing — starts from it, and every silicon bug that comes from "the simulator and the synthesis tool understood the RTL differently" is created exactly here.

In a real project the RTL designer runs synthesis long before the physical-design (PD) team does: to check that the RTL is synthesizable and latch-free, to see the area of a block before committing to an architecture, and to find the critical path early. This lab gives you that habit with open-source tools: **sv2v** (SystemVerilog to Verilog-2005), **Yosys** (synthesis), the **SkyWater 130 nm** high-density standard-cell library, and **Verilator** for gate-level simulation (GLS).

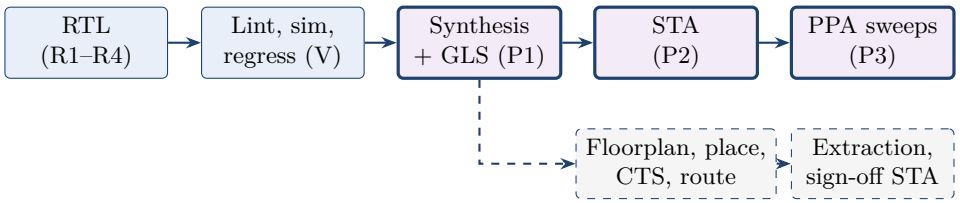
On the Synopsys cloud: Synthesis runs in **Fusion Compiler** (`fc_shell`): `make synth SYN_TOOL=fc`, the default that `env.sh` sets. Yosys and sv2v are not installed, and sv2v is not needed: Fusion Compiler reads SystemVerilog directly. Design Compiler is installed but not licensed. GLS runs in **VCS** (`SIM=vcs`, also the default) on the Fusion Compiler netlist; RTL-versus-netlist equivalence is proven with **Formality** (`make lec`, optional). The library is the generic Synopsys training library `class.db/class.lib` (`LIB_DB/LIB_STD` from `env.sh`): about 100 cells (`IV`, `ND2`, `AO2`, `FD1`, `FD2`, ...), a 5 V linear delay model, unit-less area and no Verilog cell models. Every sky130 number in Phase 3 (µm^2, ns, cell names) is therefore *not* what you will see: areas are in class units and paths are several times slower. Keep the sky130 numbers of this text as the reference for orders of magnitude; the reviewer compares *trends* (what grows, what is critical, which design is faster or smaller and by what ratio), not absolute values. When the coordinator installs SAED32 or the team PDK (`lib/README.md`), the numbers become those of a real technology.

You synthesize `seq_mul` and `comb_mul` from Lab R1 and `sync_fifo` from Lab R2 (two parameter sets), simulate the netlists with the V1 testbenches, and then deliberately break the multiplier to see mismatches between RTL and gates with your own eyes.

By the end of this lab you should be able to:

1. Explain the steps of logic synthesis (elaboration, coarse-grain optimization, generic mapping, technology mapping) and what each one can change in your design.
2. Explain what a liberty file contains and what the corner name `tt_025C_1v80` means.
3. Run sv2v and Yosys through a generic, design-independent script and `make synth`.
4. Read a Yosys log and `stat` report: find latches, removed logic, re-encoded FSMs, cell counts and area.
5. Map netlist cells and nets back to the RTL statements that created them.
6. Run the V1 testbench on the netlist with random initialization (`make gls`) and explain what GLS can and cannot find.
7. Reproduce simulation–synthesis mismatches and say which check (lint, assertion, GLS) catches each one first.

Where this lab sits. It builds on the RTL of Labs R1 and R2 (the coding rules of Lab R1 are the reason most mismatches never happen) and on the testbenches of Lab V1, which you reuse unchanged for GLS. The netlists you produce here are the input of the static timing analysis in Lab P2; the `synth` target is reused by the parameter sweeps of Lab P3 and by the pipelined multiplier of Lab T-RTL3.



Highlighted: the steps of Phase 3. Dashed: physical implementation, introduced as further reading in Lab P3.

2 What Synthesis Does

Synthesis is not one transformation but a pipeline. Knowing the stages tells you where to look when the netlist is not what you expected.

Stage	Yosys passes	What happens
Parsing and elaboration	`read_verilog`, `hierarchy`, `proc`	Parameters are resolved, generate blocks unrolled, `always` blocks converted to multiplexers, flip-flops and (if you made a mistake) latches. Initial values and delays are dropped here.
Coarse-grain optimization	`opt`, `wreduce`, `alumacc`, `share`, `fsm`, `memory`	Constant propagation, removal of unused logic, width reduction, merging of identical expressions, resource sharing of adders and multipliers, FSM extraction and re-encoding, memory inference.
Generic mapping	`techmap`, `opt`, `abc` (generic)	Word-level cells (`$add`, `$mul`, `$mux`) become single-bit gates (`$_AND_`, `$_XOR_`, `$_MUX_`) and flip-flops (`$_DFF_PNO_`, ...).
Technology mapping	`dfflibmap`, `abc -liberty`	Flip-flops are matched to library flip-flops; the combinational logic is re-optimized by ABC and covered with library cells, trading area against delay using the cell data from the liberty file.

2.1 Optimizations You Will See

Synthesis is allowed to do anything that keeps the *behavior at the ports* identical for every legal input sequence that starts from reset. That includes changes that surprise you when you look at the netlist:

- **Constant propagation.** A flip-flop whose input is always the same constant after reset is replaced by that constant. A register bit that "disappeared" usually means it could never change.
- **Dead-logic removal.** Logic that drives no output is deleted, including whole registers. If an output is unused or tied off, everything that only feeds it vanishes.
- **Width reduction.** An adder whose upper result bits are never used is shrunk; a 32-bit counter that only counts to 33 is not shrunk (synthesis does not know the value range), which is why you size counters yourself.
- **Resource sharing.** Two additions in mutually exclusive branches of a `case` can share one adder plus multiplexers.
- **FSM re-encoding.** Yosys may extract your state register and re-encode it (log: `Recoding FSM ... using 'auto' encoding`). The netlist can then contain a different number of state flip-flops than your `enum` has bits.
- **Memory mapping.** An array becomes a `$mem` cell; with no SRAM macro available it is mapped to flip-flops plus read multiplexers (the topic of Lab P3).

- **Logic restructuring.** ABC rewrites the Boolean network completely. Do not expect to find your `if` in the netlist; expect to find its function.

What synthesis does *not* do by default: it does not move flip-flops (no retiming), does not change the number of pipeline stages, and does not fix a slow architecture. Those decisions remain the RTL designer's (Labs P2 and T-RTL3).

2.2 Why sv2v

Yosys has a capable Verilog-2005 front end but only partial SystemVerilog support. sv2v converts packages, `typedef`s, packed structs, enums, interfaces and `always_ff`/`always_comb` into plain Verilog that Yosys reads reliably. Two consequences:

- The flow runs sv2v with `-DSYNTHESIS`, so everything inside `` `ifndef SYNTHESIS `` (your V1 assertions) never reaches Yosys.
- sv2v rewrites the source: names survive, but comments, line numbers and formatting do not. Remember this when you look for a synthesis pragma or a source line (experiment F).

3 Standard Cells and the Liberty File

3.1 The Library and Its Name

A standard-cell library is a set of pre-designed, pre-characterized gates of equal height that the placer can line up in rows. The name `sky130_fd_sc_hd__nand2_1` reads as: SkyWater 130 nm (`sky130`), foundry-provided (`fd`), standard cell (`sc`), high density (`hd`); cell `nand2` with drive strength `1`. Larger drive strengths (`_2`, `_4`, ...) are faster into a large load but bigger and have a higher input capacitance.

The file name `sky130_fd_sc_hd__tt_025C_1v80.lib` names the **PVT corner** the numbers were characterized at: typical–typical process (`tt`), 25 °C, 1.80 V supply. The same cells are also characterized at, for example, `ss_100C_1v60` (slow silicon, hot, low voltage: used to sign off setup) and `ff_n40C_1v95` (fast, cold, high voltage: used to sign off hold). This onboarding uses only the typical corner, so every timing number in Phase 3 is a *typical* number, optimistic compared to sign-off.

3.2 What a Liberty File Contains

A liberty (`.lib`) file is a text description of every cell, as seen by synthesis and timing tools. They never look at transistors; they look only at this file. Abridged and with illustrative values:

```
library ("sky130_fd_sc_hd__tt_025C_1v80") {
  time_unit : "1ns" ;  voltage_unit : "1V" ;  leakage_power_unit : "1nW" ;
  capacitive_load_unit (1.0, "pf") ;
  nom_process : 1.0 ;  nom_temperature : 25.0 ;  nom_voltage : 1.8 ;
  lu_table_template ("del_1_7_7") {          /* 2-D lookup table shape */
    variable_1 : "input_net_transition" ;
    variable_2 : "total_output_net_capacitance" ;
    index_1 ("0.01, 0.023, 0.053, 0.12, 0.28, 0.65, 1.5") ;
    index_2 ("0.0005, 0.0012, 0.0029, 0.0069, 0.016, 0.039, 0.093") ;
  }
  cell ("sky130_fd_sc_hd__nand2_1") {
    area : 3.7536 ;                          /* um^2 */
    cell_leakage_power : 0.0019 ;
    pin ("A") { direction : input ; capacitance : 0.0024 ; }
    pin ("B") { direction : input ; capacitance : 0.0023 ; }
```

```
    pin ("Y") {
      direction : output ;
      function  : "(!A) | (!B)" ;
      max_capacitance : 0.074 ;
      timing () {                          /* one timing arc: A -> Y */
        related_pin  : "A" ;
        timing_sense : negative_unate ;
        cell_fall ("del_1_7_7") { values ("0.024, 0.030, ...", ...) ; }
        cell_rise ("del_1_7_7") { values ("0.036, 0.047, ...", ...) ; }
        fall_transition ("del_1_7_7") { values (...) ; }
        rise_transition ("del_1_7_7") { values (...) ; }
      }
      internal_power () { related_pin : "A" ; ... }
    }
  }
  cell ("sky130_fd_sc_hd__dfrtp_1") {        /* D flip-flop, async reset */
    area : 25.024 ;
    ff ("IQ", "IQ_N") { next_state : "D" ; clocked_on : "CLK" ; clear : "!RESET_B" ; }
    pin ("D") {
      timing () { related_pin : "CLK" ; timing_type : setup_rising ; ... }
      timing () { related_pin : "CLK" ; timing_type : hold_rising ;  ... }
    }
    pin ("RESET_B") {
      timing () { related_pin : "CLK" ; timing_type : recovery_rising ; ... }
      timing () { related_pin : "CLK" ; timing_type : removal_rising ;  ... }
    }
    pin ("Q") { function : "IQ" ; timing () { related_pin : "CLK" ; timing_type :
    rising_edge ; ... } }
  }
}
```

Content	**Used for**
Units, nominal PVT	Interpreting every number; the corner of the analysis
`area`	The area reported by `stat -liberty` (cell area only, no wiring)
`function`, `ff` groups	Letting synthesis know what a cell computes, and which cells are sequential
Pin `capacitance`	The load a pin puts on the net that drives it
Timing arcs (`cell_rise`, ...)	Delay of an input-to-output arc, looked up in a table indexed by the input slew and the output load (NLDM: non-linear delay model)
Transition tables	The output slew, which becomes the input slew of the next cell
Setup, hold, recovery, removal arcs	The timing checks of sequential cells (Lab P2)
`max_capacitance`, `max_transition`	Design-rule limits; violated by an overloaded net
Leakage and internal power	Power estimation (outside the scope of the core)

Two consequences worth remembering: a cell's delay is *not* a constant (it depends on what drives it and what it drives), and a high-fanout net is slow because its capacitance is large, not because the gate is "slow". Open the real file in `lib/` and find the `nand2_1`, `dfrtp_1` and `dfxtp_1` cells; record their areas and the D-pin setup time of `dfrtp_1` at the smallest table index.

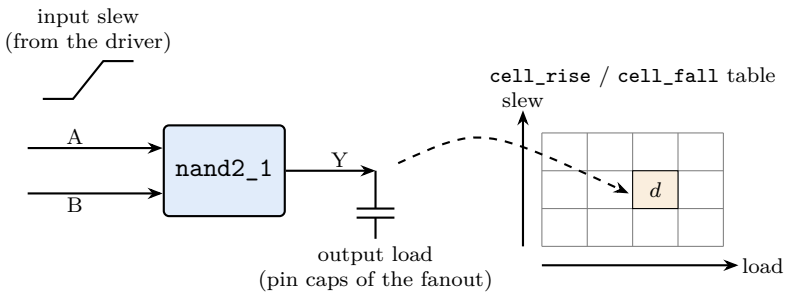


Figure 1: Non-linear delay model (NLDM): the delay of a timing arc is looked up (and interpolated) in a table indexed by the input slew and the output load; the output slew comes from a second table and becomes the input slew of the next cell.

Figure 1 is why a cell has no single delay.

On the Synopsys cloud: `class.lib` is an old-style library and makes a useful contrast: it uses the *linear* (generic CMOS) delay model — every arc has an `intrinsic_rise`/`intrinsic_fall` plus a `rise_resistance`/`fall_resistance` multiplied by the load, instead of NLDM tables — its capacitance unit is 0.1 fF (`capacitive_load_unit`), every input pin is 1 unit, and area has no unit. Do the look-up task with `ND2`, `IV`, `FD1` (D flip-flop without reset, the `dfxtp` of sky130) and `FD2` (asynchronous clear `CD`, the `dfrtp`): area, pin capacitance, the intrinsic delay and resistance of one arc, and setup/hold of `FD2` (0.85 and 0.4 ns). `class.lib` has no `recovery`/`removal` arcs (Lab P2). If SAED32 is installed, also look at one of its `.lib` files: it is NLDM like sky130.

4 The Synthesis Flow in the Repository

Figure 2 shows what `make synth` and `make gls` run and which file goes where. The make targets, the build directory and the file names are the same for both tool sets; only `SYN_TOOL` (and `SIM` for GLS) changes.

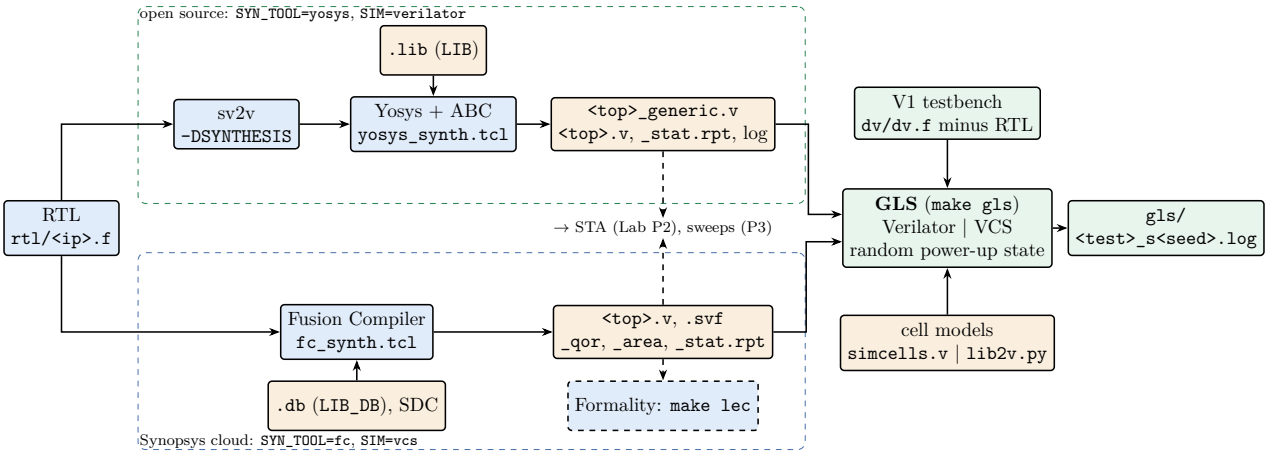


Figure 2: Synthesis and gate-level simulation flow. Top: the open-source path of this text (sv2v, Yosys, Verilator on the generic netlist). Bottom: the same targets on the Synopsys cloud (Fusion Compiler reads the SystemVerilog directly; VCS simulates the mapped netlist with Verilog models of the library cells). All outputs go to `build/<ip>/<cfg>/syn/` and `.../gls/`, named by `DUT_TOP`.

4.1 Generic Yosys Script

`flow/syn/yosys_synth.tcl` never names a design: the top, the source, the library, the output directory and the parameters come from environment variables set by `syn.mk`. The same script

synthesizes every IP of the program.

```
# flow/syn/yosys_synth.tcl -- generic synthesis script (Lab P1)
# Run by flow/mk/syn.mk as:  yosys -c yosys_synth.tcl
yosys -import
set top  $::env(DUT_TOP)
set src  $::env(SYN_SRC)        ;# single Verilog-2005 file written by sv2v
set lib  $::env(LIB)
set out  $::env(SYN_DIR)
set prms [expr {[info exists ::env(SYN_PARAMS)]  ? $::env(SYN_PARAMS)  : ""}]
set flat [expr {[info exists ::env(SYN_FLATTEN)] ? $::env(SYN_FLATTEN) : 1}]

# 1. Read and elaborate with the requested parameter values (PARAMS)
read_verilog -defer $src
set chp {}
foreach p $prms { lassign [split $p =] n v; lappend chp -chparam $n $v }
hierarchy -check -top $top {*}$chp
yosys rename -top $top          ;# plain module name, whatever the PARAMS

# 2. Generic synthesis (Yosys internal gates)
set sargs [list -top $top]
if {$flat} { lappend sargs -flatten }
synth {*}$sargs
tee -o $out/${top}_stat_generic.rpt stat
write_verilog -noattr $out/${top}_generic.v

# 3. Technology mapping to SkyWater 130 nm
dfflibmap -liberty $lib
abc -liberty $lib
setundef -zero
hilomap -singleton -hicell sky130_fd_sc_hd__conb_1 HI -locell sky130_fd_sc_hd__conb_1 LO
splitnets
opt_clean -purge
check
tee -o $out/${top}_stat.rpt stat -liberty $lib
write_verilog -noattr -noexpr -nohex -nodec $out/${top}.v
write_verilog $out/${top}_src.v ;# keeps (* src *) attributes, for mapping to RTL
```

Notes: `rename` and `proc` clash with Tcl built-ins, so they are called as `yosys rename`. `hierarchy -chparam` sets parameters of the top before elaboration (sv2v has no `-G`). `hilomap` replaces constant drivers by tie cells, `splitnets` turns buses into single-bit nets (OpenSTA and many back-end tools prefer this). `SYN_FLATTEN=0` keeps the hierarchy; Lab P3 uses it for per-module area.

On the Synopsys cloud: The Fusion Compiler equivalent is `flow/syn/fc_synth.tcl`, equally design-independent (same environment variables plus `LIB_DB`, `SYN_SDC` and `SYN_FLAT`, the flat file list). Its steps: `set_non_physical_mode` (logic synthesis only, zero wire load); `create_lib` with the reference libraries; `analyze -format sverilog -define SYNTHESIS` of the file list (no sv2v); `elaborate <top> -parameters "WIDTH=8,DEPTH=16"` (the equivalent of `hierarchy -chparam`) followed by `set_top_module` and `rename_module` to the plain top name (elaborate calls it `sync_fifo_WIDTH8_DEPTH16`); the SDC of Lab P2 at `SYN_CLK_PERIOD` (default 10 ns; Fusion Compiler is timing-driven, Yosys is not); `set_scan_element false` on all registers (otherwise every flip-flop becomes a scan flip-flop, ready for DFT); `ungroup_cells -all -flatten` unless `SYN_FLATTEN=0`; `compile_logical`; `report_qor`, `report_area`, `report_reference` (the `stat` equivalent), `report_timing`; `write_verilog`; and an SVF file for Formality. Fusion Compiler only links libraries in NDM format with a technology and a physical view of every cell; with nothing but `class.db`, `create_lib -ref_libs class.db` fails (`LIB-117`, `NDM-020`), and a library built from dummy macro frames fails in compile (`OPT-1006`: cannot find buffer, inverter and a basic gate). The flow therefore gives `create_lib` a minimal technology file (`flow/syn/fc_bringup.tf`) and a placeholder LEF generated from the `.lib` (`flow/scripts/lib2lef.py`); Fusion Compiler then builds the NDM itself. Timing comes from the `.db`; the area that Fusion Compiler reports is the frame area (width × row height), so the placeholder widths are the Liberty `area` divided by the row height, exactly (a first version rounded them up to whole sites and to one site per pin, and inflated every area by 5–20 %: check that `report_reference` shows the unit areas of the `.lib`). With a real PDK set `LIB_NDM` and `LIB_TF` instead (`lib/README.md`).

4.2 `syn.mk`: the `synth` and `gls` Targets

```
# flow/mk/syn.mk -- logic synthesis and gate-level simulation (Lab P1)
# From the IP Makefile: IP DUT_TOP RTL_FLIST TB_FLIST TB_TOP PARAMS DEFINES
# From common.mk:       REPO_ROOT BUILD_DIR LIB_DIR SEED TEST PLUSARGS
LIB         ?= $(LIB_DIR)/sky130_fd_sc_hd__tt_025C_1v80.lib
SYN_FLATTEN ?= 1
SYN_DIR     := $(BUILD_DIR)/syn
GLS_DIR     := $(BUILD_DIR)/gls
FLIST_EXP   := $(REPO_ROOT)/flow/scripts/flist_expand.sh

RTL_ITEMS   := $(shell $(FLIST_EXP) $(RTL_FLIST))
RTL_SRCS    := $(filter-out +%,$(RTL_ITEMS))
RTL_INCS    := $(patsubst +incdir+%,-I%,$(filter +incdir+%,$(RTL_ITEMS)))
SV2V_OUT    := $(SYN_DIR)/$(DUT_TOP)_sv2v.v
SYN_LOG     := $(SYN_DIR)/$(DUT_TOP)_yosys.log
# sky130 netlist (STA, Lab P2) and generic netlist (GLS)
NETLIST     := $(SYN_DIR)/$(DUT_TOP).v
NETLIST_GEN := $(SYN_DIR)/$(DUT_TOP)_generic.v

.PHONY: synth gls
synth: $(NETLIST)

$(NETLIST): $(RTL_SRCS) $(REPO_ROOT)/flow/syn/yosys_synth.tcl
        @mkdir -p $(SYN_DIR)
        sv2v -DSYNTHESIS $(addprefix -D,$(DEFINES)) $(RTL_INCS) \
          $(RTL_SRCS) > $(SV2V_OUT)
        DUT_TOP=$(DUT_TOP) SYN_SRC=$(SV2V_OUT) SYN_DIR=$(SYN_DIR) LIB=$(LIB) \
        SYN_PARAMS="$(PARAMS)" SYN_FLATTEN=$(SYN_FLATTEN) \
          yosys -q -l $(SYN_LOG) -c $(REPO_ROOT)/flow/syn/yosys_synth.tcl
        @if grep -n 'Latch inferred' $(SYN_LOG); then \
           echo "ERROR: latch inferred, see $(SYN_LOG)"; exit 1; fi
        @grep -A40 'Number of cells' $(SYN_DIR)/$(DUT_TOP)_stat.rpt
```

```
# --- GLS: the V testbench on the generic netlist -------------------
TB_ITEMS  := $(shell $(FLIST_EXP) $(TB_FLIST))
# keep packages and interfaces (the TB needs them), drop the RTL modules
GLS_DROP  := $(filter-out %_pkg.sv %_if.sv,$(RTL_SRCS))
GLS_SRCS  := $(filter-out $(GLS_DROP),$(filter-out +%,$(TB_ITEMS)))
SIMCELLS  := $(shell yosys-config --datdir)/simcells.v
GLS_FLAGS := --binary --timing -Wno-fatal -Wno-PINNOTFOUND \
             --x-assign unique --x-initial unique +define+GLS
GLS_LOG   := $(GLS_DIR)/$(TEST)_s$(SEED).log

gls: $(NETLIST)
        @mkdir -p $(GLS_DIR)
        verilator $(GLS_FLAGS) $(addprefix -G,$(PARAMS)) $(addprefix +define+,$(DEFINES)
    ) \
          $(filter +incdir+%,$(TB_ITEMS)) $(GLS_SRCS) $(NETLIST_GEN) $(SIMCELLS) \
          --top-module $(TB_TOP) -Mdir $(GLS_DIR)/obj
        $(GLS_DIR)/obj/V$(TB_TOP) +verilator+rand+reset+2 +verilator+seed+$(SEED) \
          +TEST=$(TEST) +SEED=$(SEED) $(PLUSARGS) | tee $(GLS_LOG)
        @grep -q 'TEST PASSED' $(GLS_LOG)
```

Design decisions, each worth defending in the review:

- **One build directory per configuration.** The netlist is written to `build/<ip>/<cfg>/syn/`, so a netlist synthesized with `DEPTH=16` can never be simulated by a testbench compiled with `DEPTH=5`: `gls` uses the same `PARAMS` for the netlist and the testbench top.
- **Outputs named by `DUT_TOP`**, so `seq_mul` and `comb_mul` (same IP, same configuration) do not overwrite each other.
- **Latches fail the build.** Yosys prints `Latch inferred for signal ...` for every latch; none of our designs needs one.
- `-Wno-PINNOTFOUND`: the netlist module has no parameters any more (they were resolved at synthesis), but the testbench still instantiates it with `#(.DEPTH(DEPTH))`.
- `+define+GLS`: see Section 7.3.

On the Synopsys cloud: `flow/mk/syn.mk` dispatches on `SYN_TOOL` the way `lint.mk` dispatches on `LINT_TOOL`; the `gls` recipe dispatches on `SIM`, so every combination works. With `SYN_TOOL=fc` it runs `fc_shell -f fc_synth.tcl < /dev/null` (without `/dev/null` `fc_shell` waits at its prompt after an error) and fails unless the script reached its last line; latches fail the build (`all_registers -level_sensitive`). With `SIM=vcs`, `gls` compiles the testbench without the RTL modules, the netlist and the cell models with `vcs -sverilog +define+GLS +vcs+initreg+random` and runs `simv +vcs+initreg+random +ntb_random_seed=<SEED>`: every `reg` gets a random power-up value derived from the seed (a number in place of `random`, as in `+vcs+initreg+1`, means *all ones*, not a seed). `GLS_INIT=x` keeps VCS's four-state X instead, a second and stricter way to find missing resets. The VCS message for the parameter override of the parameter-less netlist module is `Warning-[AOUP] Attempt to override undefined parameter` (Verilator: `PINNOTFOUND`); it is expected. The cell models are `LIB_VERILOG` when the library provides them (SAED32, sky130), otherwise generated from the `.lib` by `flow/scripts/lib2v.py` into `build/lib/`.

5 Reading the Yosys Log and Statistics

A clean exit code means only that Yosys did not crash. The log is where synthesis tells you what it understood. Search `build/<ip>/<cfg>/syn/<top>_yosys.log` for the messages below and explain every one you find.

Log message	Meaning and action
`Latch inferred for signal`	An `always_comb` does not assign a variable on every path (R1 rule 1). Always a bug here.
`No latch inferred for signal`	Normal: a combinational variable was recognized correctly.
`Replacing memory ... with list of registers`	An array could not stay a `$mem` cell (for example, it is reset or read asynchronously). Expected for small arrays; note it.
`Recoding FSM ... using 'auto' encoding`	Your FSM was extracted and re-encoded; the mapping table follows in the log.
`Removing unused output` / `Removed N unused cells`	Logic without a path to an output was deleted. Make sure you meant it.
`Resizing cell port`	A width mismatch between a cell and its connection (R1 rule 12). Find the RTL statement.
`multiple conflicting drivers`	One net driven from two places (R1 rule 4). Always a bug.
`is used but has no driver`	An undriven signal; synthesis will tie it to a constant or leave it floating.
`found and reported N problems` (from `check`)	Must be 0 for the final netlist.

On the Synopsys cloud: The Fusion Compiler log is `<top>_fc.log`. The `elaborate` step prints an *Inferred memory devices* table with one line per register (type Flip-flop or **Latch**, width, set/reset, synchronous or asynchronous) — the first place to check your flip-flop count and to see a latch. Every message carries a tag (`ELAB-...`, `VER-...`, `OPT-...`); `make synth` prints the list of unique warning tags, and `man <TAG>` in `fc_shell` explains each. Typical ones in this flow: `CSTR-011` (`set_max_fanout` is not supported by Fusion Compiler, harmless), `CGT-3300` (no clock-gating cell in the library), `OPT-070` (no default max transition), `VER-941` (a `\0`-style escape in a `$error` string). Constant or unloaded registers are listed with their removal reason; `check_design` and `check_timing` are in `<top>_check.rpt`.

5.1 The `stat` Report

The generic report counts Yosys internal cells; the sky130 report counts library cells and sums their `area` attributes. Example for `seq_mul` with `XLEN=32` (illustrative numbers; your RTL gives different ones, and the exact format varies with the Yosys version):

```
=== seq_mul ===   (generic: <top>_stat_generic.rpt)
   Number of cells:                904
     $_ANDNOT_                     214
     $_AND_                         38
     $_DFFE_PN0P_                    8
     $_DFFE_PP_                    128
     $_DFF_PN0_                      3
     $_DFF_P_                       32
     $_MUX_                         97
     $_OR_                         131
     $_XOR_                         84
     ...

=== seq_mul ===   (sky130: <top>_stat.rpt)
   Number of cells:                906
     sky130_fd_sc_hd__dfxtp_1      160
     sky130_fd_sc_hd__nand2_1      121
     sky130_fd_sc_hd__mux2_1        96
     sky130_fd_sc_hd__a21oi_1       88
     sky130_fd_sc_hd__o21ai_0       74
```

```
    sky130_fd_sc_hd__xnor2_1       70
    ...
    sky130_fd_sc_hd__dfrtp_1       11
    sky130_fd_sc_hd__conb_1         1
  Chip area for module '\seq_mul': 8300.461000
    of which used for sequential elements: 3478.336000 (41.90%)
```

How to read it:

- **Flip-flops first.** Count every `$_DFF*` / `df*` cell and compare with the registers you intended (R1 review question "point to every flip-flop"). Here $3 + 8 + 128 + 32 = 171$ generic flip-flops become $160 + 11 = 171$ library flip-flops. The 11 `dfrtp` (reset) cells are the control registers; the 160 `dfxtp` (no reset) are datapath registers that R1 rule 6 allowed to skip reset. Resetting everything would cost about $5\,\mu\text{m}^2$ per bit here.
- **Enable flip-flops.** `$_DFFE_PP_` are flip-flops with a load enable. This library version is mapped as a plain flip-flop plus a `mux2` feeding back Q: 128 enable flip-flops, $96 + 32 = 128$ `mux2`/`mux2i` cells.
- **Area is cell area only.** No wires, no placement utilization. A placed block is typically 1.4–2× larger than the sum of its cell areas.
- **Sequential fraction.** Above 40 % for a multiplier that computes one partial product per cycle; the combinational `comb_mul` is the opposite case (Lab P2).

On the Synopsys cloud: `<top>_stat.rpt` is `report_reference`: one line per library cell with unit area, count and total area; `<top>_area.rpt` splits combinational, buffer/inverter and non-combinational (sequential) area; `<top>_summary.txt` has the numbers of this section (cells, flops, area, sequential fraction) in a form that the sweep of Lab P3 reads. Fusion Compiler may implement a flip-flop that resets to 0 with a preset flip-flop (`FD4`) and take its inverted output `QN`: count it as a flip-flop of the same register, and check the total against your RTL.

6 Mapping the Netlist Back to RTL

A timing report or a GLS failure names netlist objects; you must find the RTL behind them. Instance names like `_0835_` are anonymous, but three things survive synthesis:

1. **Register output nets.** The net on the Q pin of a flip-flop keeps the name of the RTL variable (`count_q[3]`, `state_q[0]`). Search the netlist for `.Q(count_q[3])` to find the flip-flop instance.
2. **Port names.** Unchanged (struct ports become plain vectors; interface ports become `<port>_<signal>`).
3. **Source attributes.** `<top>_src.v` keeps `(* src = "seq_mul_sv2v.v:87.5-93.8" *)` on many cells. The location points into the sv2v output, which keeps your names, so one more search finds the RTL statement.

For small blocks, the Yosys `show` command draws the generic netlist of one register's input cone, for example `show -format svg w:count_q %ci*:-$_DFF_PN0_` (run interactively on a copy of the script). **Task:** for `seq_mul`, identify in the sky130 netlist (a) the state flip-flops and whether the FSM was re-encoded, (b) the iteration counter and its incrementer (look for `xor2`/`ha` cells feeding `count_q`), (c) at least three cells of the partial-product adder, (d) the operand load multiplexers. Record each with its instance name, cell type and RTL line. Lab P2 asks you to do the same for the cells of the critical path.

On the Synopsys cloud: Fusion Compiler names every flip-flop after its RTL variable: `prod_q_reg_62_` is bit 62 of `prod_q` (`change_names -rules verilog` turns `prod_q_reg[62]` into a plain identifier); the combinational cells are anonymous (`ctmi_*`, nets `ctmn_*`). There is no `src` attribute; instead open the netlist in Verdi (`verdi -sv <netlist> <cell models>`) and trace the fan-in cone of a register in the schematic, or use `all_fanin -to prod_q_reg_62_/D` in `pt_shell` (Lab P2).

7 Gate-Level Simulation

7.1 What GLS Finds, and What It Does Not

Test term — Gate-level simulation (GLS). The verification testbench run on the synthesized netlist instead of the RTL, with the same tests and seeds. A pass proves that, for the stimulus the tests apply, synthesis built the hardware the RTL simulation showed: no simulation–synthesis mismatch, no reset that only worked thanks to an initializer, no X that the RTL resolved optimistically. It is run here because P1 is the first lab with a netlist; it proves nothing about timing (zero delay) or about untested behavior.

GLS simulates the netlist with the same testbench and the same tests as the RTL. If the results differ, synthesis built different hardware from what the simulator executed: a simulation–synthesis mismatch.

GLS finds	**GLS does not find**
Initializers and `initial` blocks that silicon ignores (R1 rule 6)	Timing problems: our GLS is zero-delay; timing is the job of STA (Lab P2)
Missing resets, when combined with random initialization	Bugs the testbench does not exercise (GLS is only as good as V1)
X-optimism in the RTL that synthesis resolved differently (rule 7)	Mismatches when synthesis happens to match the simulator for your stimulus
Code removed by pragmas or `` `ifdef SYNTHESIS ``	Problems introduced after synthesis (scan insertion, ECOs)
Sensitivity and ordering problems in combinational blocks (rules 2, 5)	Clock-domain crossing problems (Lab T-RTL1)

In industry, zero-delay GLS is run on a few tests of every netlist release, and *timing-annotated* GLS (with an SDF file written by STA after place and route) on a few more to catch reset and X-propagation problems; formal equivalence checking (RTL versus netlist) is what actually proves the netlist correct. Neither is part of this lab.

Test term — Formal equivalence check. Not a simulation: a tool proves that the netlist computes the same function as the RTL for *every* input sequence, register by register (after matching the registers of both sides). A pass is complete where GLS is only as good as its tests; it does not replace GLS for initialization and X problems, which are outside the RTL semantics both sides are compared under.

On the Synopsys cloud: On the cloud the equivalence check is available: `make lec` runs Formality (`flow/syn/fm_lec.tcl`): `read_db` of the library, `read_sverilog -r` of the RTL with `-define SYNTHESIS`, `set_top -parameter` with `PARAMS`, `read_verilog -i` of the netlist, the SVF guidance written by Fusion Compiler (register merging, inverted and constant registers), `match`, `verify`. It passes with `lec: SUCCEEDED`. Optional in this lab; try it on `seq_mul` and on one mismatch experiment.

7.2 Which Netlist

`make gls` simulates the *generic* netlist together with `simcells.v`, the Verilog models of Yosys internal cells shipped with Yosys. Every decision that can create a mismatch — dropping initializers and delays, honoring or ignoring pragmas, resolving X, interpreting `always_comb`, optimizing unreachable states — has already happened at this point; technology mapping afterwards is a logic-preserving re-implementation. Simulating the sky130 netlist needs the foundry's Verilog cell models, which use user-defined primitives that Verilator supports only partly; it is optional (see Section 9).

On the Synopsys cloud: Fusion Compiler writes only the mapped netlist, so on the cloud GLS always simulates the technology netlist: the optional task of Section 9 is the default there. With `class.db` the cell models are the zero-delay models that `flow/scripts/lib2v.py` writes from the `function`, `ff` and `latch` groups of `class.lib`; VCS also simulates foundry models with user-defined primitives and `specify` blocks (SAED32: `LIB_VERILOG`). The mismatch-creating steps have all happened in Fusion Compiler's elaboration, just as in Yosys's.

7.3 Reusing the V1 Testbench

GLS reuses `hw/ip/<ip>/dv/` unchanged in behavior.

Test term — Regression. The whole set of tests × seeds × parameter sets of `dv/regress.list`, run by one command with a pass/fail summary. A pass proves that nothing that used to work is broken. In this lab the regression is re-run on the netlist (with fewer transactions), so the same tests that signed off the RTL now sign off the gates.

The V1 test names you run on the netlist, and what each one is meant to show: `smoke` (the R1 smoke requests: the netlist is alive and connected); `corners` (directed corner operands — 0, 1, all ones, most negative, powers of two — for every operation); `back_to_back`, `hold` and `busy_valid` (handshake sequences: results held under back-pressure, no acceptance while busy); `random` (random operands and timing, thousands of transactions, many seeds); `exhaustive` (every operand pair for every operation at `XLEN=8`: complete for that width); for `sync_fifo` `fill_drain` (to full and back to empty: the boundary flags), `simultaneous` (push and pop in the same cycle at every level) and `random`. Three things need attention:

- **Same tests, same seeds.** Run the tests of your V1 `dv/regress.list` on the netlist (fewer transactions are fine: GLS is slower). For a given `TEST`, `SEED` and `PARAMS`, the RTL and GLS logs must report the same number of checked transactions and both `TEST PASSED`.
- **Bound assertions.** Port-level assertions (handshake, stability) still work on the netlist, because port names survive. Assertions that read internal signals (`count`, `state`) cannot bind: those signals no longer exist. Guard such bind statements with `` `ifndef GLS `` in `dv/sva/<ip>_bind.sv`. This is a build change, not a new check, and it shows you which of your assertions are implementation-independent.
- **Random initialization.** `--x-initial unique` and `+verilator+rand+reset+2` give every flip-flop a random power-up value, as in silicon. Verilator is two-state: without this, every flip-flop starts at 0 and a missing reset stays invisible. Run at least five seeds.

```
cd $REPO_ROOT/hw/ip/seq_mul
make synth                                  # build/seq_mul/default/syn/
make gls TEST=random SEED=1 PLUSARGS="+NUM_TXN=2000"
make synth gls DEFINES=MUL_XLEN=8 TEST=exhaustive SEED=3
make synth DUT_TOP=comb_mul                 # synthesis only, no GLS
cd ../sync_fifo
make synth gls PARAMS="WIDTH=8 DEPTH=16" TEST=random SEED=5
make synth gls PARAMS="WIDTH=32 DEPTH=5" TEST=random SEED=5
```

On the Synopsys cloud: The same commands, unchanged: `env.sh` selects `SYN_TOOL=fc` and `SIM=vcs`. Additionally `make lec` (Formality), `make synth SYN_CLK_PERIOD=4` (timing-driven synthesis for a 4 ns clock: a different netlist; the default 10 ns keeps the netlist fixed for the STA of Lab P2), `make gls GLS_INIT=x` (power-up X instead of random values) and `make waves`-style debugging of a failing GLS run with `SIMV_FLAGS`/`VCS_FLAGS` (`-kdb -debug_access+all`). Synthesis of a 32-bit multiplier takes one to a few minutes in Fusion Compiler.

8 Mismatch Experiment

Reading the R1 rules is not enough; you must see a mismatch happen.

Test term — Fault injection (mismatch experiment). A known fault is put into the design on purpose and the checks are run to see which one catches it. It tests the checks, not the design: a check that never fires on a real fault proves nothing (the bug-hunt idea of Lab V1). Here the faults are simulation–synthesis mismatches, and the question is which check — lint, an assertion or GLS — sees each one first.

Create a branch `exp/p1-mismatch` from your P1 branch (it is never merged), edit `hw/ip/seq_mul/rtl/seq_mul.sv` directly and introduce *one fault at a time* (one commit per experiment, reverted before the next). For each fault, run the RTL simulation and the gate-level simulation, both with random initialization and the same seeds, and compare them.

```
make sim TEST=random SEED=11      # RTL; V1 flow, random init
make synth gls TEST=random SEED=11
make lint                          # what did lint say about the fault?
```

If your `sim_verilator.mk` does not yet compile with `--x-assign unique --x-initial unique` and run with `+verilator+rand+reset+2`, add it now for `sim` as well: it is a flow fix, and without it the RTL side of experiments A and E cannot show the silicon behavior.

Experiments A and B are required; choose at least two more.

Experiment	Fault to introduce	R1 rule
A (required)	Remove the reset of the state register and use a declaration initializer instead	6
B (required)	In an `always_comb` block, read a temporary variable before assigning it	5
C	Use blocking assignments in two `always_ff` blocks that feed each other	3
D	Add a `#` delay to a combinational assignment	9
E	Remove the reset from a control flop that drives an `if` condition	7
F	Wrap part of the next-state logic in `translate_off` / `translate_on`	9
G	Write the multiplication as a `for` loop in `always_comb` and compare its synthesis result with yours	10

For each experiment, record in `hw/ip/seq_mul/doc/ppa.md`:

1. The exact code change (a short diff).
2. The RTL simulation result and the gate-level simulation result, with a waveform screenshot where they differ.
3. Which result matches the real hardware, and why.
4. Which check would have caught it first: a lint warning, an assertion (V1), or gate-level simulation.

If an experiment shows no difference, that is a valid result. Explain why it did not appear and under what conditions it would. Tool-specific hints (verify them; do not copy them as results):

- **A:** the RTL starts in the initializer's state on every seed; the netlist has no initial value, so the state flip-flops start random. Watch the first cycles after reset release.
- **B:** Verilator orders statements inside `always_comb` and reports `ALWCOMBORDER`; read what lint said before you look at waveforms.
- **D:** `-Wall` reports `ASSIGNDLY`; with `--timing` the RTL simulation honors the delay, synthesis drops it. Does the testbench sample before or after the delayed value settles?
- **F:** sv2v does not preserve comments, so check whether the pragma even reaches Yosys in this flow. Explain what a flow that honors it would build, and why `` `ifndef SYNTHESIS `` is the only safe way to hide code from synthesis.
- **G:** compare `stat` (cells, area, flip-flops) with your sequential design; this is a preview of the `seq_mul`/`comb_mul` comparison in Lab P2.

On the Synopsys cloud: Differences that change the experiments with VCS and Fusion Compiler: VCS is four-state, so an uninitialized register is X, not 0 — the RTL simulation of experiment A still starts in the initializer's state (simulators honor initializers), while the GLS run starts random (`+vcs+initreg+random`) or X (`GLS_INIT=x`). VCS rejects a declaration initializer on a variable written by `always_ff` (`Error-[ICPD_INIT] Illegal combination of drivers`: the standard forbids a second driver), so for experiment A write the state register with a plain `always @(posedge clk)`; the compile error is itself a result worth recording. Fusion Compiler reads the source itself, so the comments survive: `// synopsys translate_off` in experiment F *is* honored and the wrapped logic is removed from the netlist — the mismatch that the sv2v flow hides becomes visible in GLS. Run `make lint` (SpyGlass) for every experiment and record which rule reports the fault (delays, blocking assignments in sequential blocks, read-before-write in combinational blocks); `make lec` reports the experiments that change the function as failing compare points.

9 Requirements / Tasks

1. Write `flow/syn/yosys_synth.tcl` and `flow/mk/syn.mk` as specified (adapt to your tool versions, keep them design-independent) and include `syn.mk` in the Makefiles of `seq_mul` and `sync_fifo`.
2. In the liberty file, look up area, pin capacitance and one delay table entry of `nand2_1`, `inv_1`, `dfxtp_1` and `dfrtp_1`; record setup and hold of `dfrtp_1`.
3. Synthesize `seq_mul` (default and `MUL_XLEN=8`), `comb_mul` (default) and `sync_fifo` (`WIDTH=8 DEPTH=16` and `WIDTH=32 DEPTH=5`). Zero latches; every log warning explained in `doc/ppa.md`.
4. For each netlist record cells, flip-flops, area and sequential fraction, and check the flip-flop count against your RTL.
5. Map the netlist of `seq_mul` back to the RTL (Section 6).
6. Run GLS of `seq_mul` (`random` and `exhaustive` with `MUL_XLEN=8`) and of `sync_fifo` (all V1 tests, both parameter sets), each with at least five seeds. All pass, with the same transaction counts as RTL.
7. Perform mismatch experiments A, B and at least two of C–G.
8. Optional: GLS of the sky130 netlist with the foundry cell models (Icarus Verilog or a Verilator-compatible model set from the coordinator); compare with the generic GLS. On the Synopsys cloud GLS of the technology netlist is the default; the optional task there is `make lec`.
9. `make -C $REPO_ROOT check-all` still passes: P1 changes no RTL and no earlier target.

10 Repository and Flow

Path	Change in this lab
`flow/syn/yosys_synth.tcl`	New: generic Yosys script
`flow/mk/syn.mk`	New: `synth`, `gls`
`hw/ip/seq_mul/Makefile`, `hw/ip/sync_fifo/Makefile`	Add `include $(REPO_ROOT)/flow/mk/syn.mk`
`hw/ip/<ip>/dv/sva/<ip>_bind.sv`	Internal-signal binds guarded by `` `ifndef GLS ``
`hw/ip/seq_mul/doc/ppa.md`, `hw/ip/sync_fifo/doc/ppa.md`	New: synthesis results, GLS results, mismatch experiments
`hw/ip/<ip>/README.md`	Add `synth`/`gls` commands and a results line
`build/<ip>/<cfg>/syn/`, `.../gls/`	Generated: sv2v output, logs, stat reports, netlists

New targets and variables:

Target / variable	Function
`synth`	sv2v (`-DSYNTHESIS`) + Yosys; writes `<top>_generic.v`, `<top>.v` (sky130), `<top>_src.v`, stat reports and log; fails on latches
`gls`	V testbench (`TB_FLIST`, `TB_TOP`) on the generic netlist with random init; exit code $\neq 0$ unless `TEST PASSED`
`LIB`	Liberty file; default `$(LIB_DIR)/sky130_fd_sc_hd__tt_025C_1v80.lib`
`SYN_FLATTEN`	`1` (default) flattens the hierarchy; `0` keeps it (Lab P3)
`DUT_TOP`	Existing; override to synthesize another module of the IP (`comb_mul`)

On the Synopsys cloud: Files and variables of the Fusion Compiler path, next to the open-source ones: `flow/syn/fc_synth.tcl` (synthesis), `flow/syn/fc_bringup.tf` and `flow/scripts/lib2lef.py` (bring-up library for a `.db` without NDM), `flow/scripts/lib2v.py` (cell models for GLS), `flow/syn/fm_lec.tcl` (`make lec`). Variables: `SYN_TOOL` (`yosys|fc`), `LIB_DB` (instead of `LIB`), `SYN_CLK_PERIOD`, `LIB_NDM/LIB_TF/LIB_VERILOG` for a real PDK, `GLS_INIT`. Generated: `<top>_fc.log`, `<top>.v`, `<top>.svf`, `<top>_{qor,area,stat,timing,check}.rpt`, `<top>_summary.txt` in `syn/`; `vcs/` and the logs in `gls/`; the cell models and the placeholder LEF in `build/lib/`.

Generated files for one run (`seq_mul`, default configuration):

```
build/seq_mul/default/syn/
  seq_mul_sv2v.v            Verilog-2005 from sv2v (-DSYNTHESIS)
  seq_mul_yosys.log         full Yosys log
  seq_mul_stat_generic.rpt  generic cell statistics
  seq_mul_stat.rpt          sky130 cell statistics and area
  seq_mul_generic.v         netlist of Yosys internal cells   (GLS)
  seq_mul.v                 sky130 netlist                    (STA, Lab P2)
  seq_mul_src.v             sky130 netlist with src attributes
build/seq_mul/default/gls/
  obj/                      Verilator model
  random_s1.log
```

11 Deliverables

Merge request `lab/p1-synthesis` (tag `p1-done`) containing:

- `flow/syn/yosys_synth.tcl`, `flow/mk/syn.mk`, the updated IP Makefiles and bind files.
- `hw/ip/seq_mul/doc/ppa.md`: synthesis table (cells, flip-flops, area, sequential fraction) for `seq_mul` (both widths) and `comb_mul`; explanation of every log warning; the netlist-to-RTL mapping table; GLS results (tests, seeds, transaction counts RTL versus GLS); the mismatch experiments with diffs, results, screenshots and answers.
- `hw/ip/sync_fifo/doc/ppa.md`: synthesis table for both parameter sets with a short comment on what grows with `WIDTH` and `DEPTH`; GLS results.
- README of both IPs: how to run `synth` and `gls`.
- The `exp/p1-mismatch` branch pushed (not merged) and referenced from `ppa.md`.

Every number in `ppa.md` must be reproducible with the documented `make` command from a clean checkout.

12 Assessment

Criterion	**Weight**
Generic flow: `yosys_synth.tcl`, `syn.mk`, no design names, reproducible	20%
Synthesis results: latch-free, log warnings explained, correct flip-flop accounting	15%
Liberty and synthesis understanding (liberty task, optimizations, review)	10%
Netlist-to-RTL mapping	10%
Gate-level simulation: all tests and seeds pass, RTL/GLS comparison	20%
Mismatch experiments: results, root cause, which check catches it	20%
Documentation (`ppa.md`, README)	5%
Total	100%

13 Review Questions

1. Name the four stages of synthesis and one transformation in each that can make the netlist look different from your RTL.
2. What does `tt_025C_1v80` mean, and which corner would you use to sign off setup? Hold?
3. Why is the delay of a `nand2_1` not a single number in the liberty file? What two quantities index the table?
4. Your `stat` shows 3 fewer flip-flops than you expected. List three possible reasons and how you would confirm each.
5. Why does the flow run sv2v with `-DSYNTHESIS`, and what would happen to your V1 assertions without it?
6. Why does the GLS use the generic netlist, and what kind of bug could only the sky130 netlist simulation find?

7. Why is random initialization essential for GLS in Verilator? What would experiment A show without it?
8. Why is the gate-level result more trustworthy than the RTL result in experiment A?
9. Which of your V1 assertions could not be bound to the netlist, and what does that tell you about where to write assertions?
10. A timing report names the flip-flop `_0835_`. How do you find the RTL variable it implements?

14 References

Simulation–synthesis mismatches (partly written for Verilog; the ideas apply to SystemVerilog):

- Clifford E. Cummings, RTL Coding Styles That Yield Simulation and Synthesis Mismatches, SNUG 1999.
- Circuit Quest, The Sim/Synth Mismatch Checklist.
- VLSI Mentor, Gate-Level Simulation: What Synthesis Changes — and Why It Matters.
- VLSI-Design, RTL coding styles that lead to pre- and post-synthesis simulation mismatch.

Tools and libraries:

- Yosys manual: `synth`, `hierarchy`, `dfflibmap`, `abc`, `stat`, `show`, and the chapter on the internal cell library (`simcells.v`).
- sv2v README: supported constructs and options.
- Verilator User's Guide: `--x-assign`, `--x-initial`, `+verilator+rand+reset`, warning codes `ALWCOMBORDER`, `ASSIGNDLY`, `PINNOTFOUND`.
- SkyWater SKY130 PDK documentation: `sky130_fd_sc_hd` library, cell naming, corners.
- Liberty format: the open Liberty Reference Manual (opensourceliberty.org).

Background: Weste and Harris, *CMOS VLSI Design*, chapters on standard cells and delay models.